



Что мы будем делать

Сейчас мы соберём настоящую мультиагентную цепочку в Hermes. У нас будет небольшой Python-проект, использующий Pydantic первой версии. Наша задача будет заключаться в обновлении зависимости до Pydantic 2 и адаптации существующего кода.

Но главное здесь не сама миграция Pydantic. Если бы нам нужно было просто обновить несколько строк, один агент справился бы с этим без проблем. Нам важно специально разделить работу между тремя исполнителями.

Первый субагент будет Planner. Он получит проект, изучит его и составит план миграции, но ничего менять не будет. Второй субагент будет Coder. Он получит исходную задачу вместе с результатом Planner, внесёт изменения и запустит проверки. Третий субагент будет Reviewer. Он начнёт работу с отдельным контекстом, посмотрит уже изменённый проект и независимо проверит результат.

Это действительно разные экземпляры агентов, а не просто три ответа одного чата. Hermes создаёт субагентов через `delegate_task`; каждый дочерний агент получает собственный контекст и отдельную терминальную сессию, а основному агенту возвращается итог его работы. История разговора родителя автоматически дочернему агенту не передаётся, поэтому контекст для каждого этапа необходимо формировать явно.

Подготавливаем Hermes Desktop

Я открываю Hermes Desktop. Если приложение уже установлено, его можно запустить обычным способом или командой:

```
hermes desktop
```

В центре окна находится наш основной чат. Справа Hermes Desktop позволяет открыть файловый браузер и терминал. Терминал можно показать сочетанием `Ctrl + ```. Во время работы субагентов над полем ввода появляется отдельная панель `Subagents`, где можно видеть активных исполнителей, время их работы и текущую активность. Эту панель мы обязательно будем использовать во время практики.

Перед началом я проверяю, что основному агенту доступны необходимые инструменты. Нам потребуются работа с файлами, терминалом и делегирование.

В поле чата я ввожу:

```
/tools list
```

Я ищу среди доступных инструментов `delegate_task`, `terminal`, `read_file`, `write_file`, `patch` и `search_files`.

Если они отключены, я могу открыть настройки Hermes Desktop и проверить конфигурацию инструментов либо включить их в текущей сессии. Для разработки особенно удобно использовать toolset `coding`: он уже включает работу с файлами, терминалом и делегирование субагентам.

Нам принципиально нужен `delegate_task`. Если его нет, никакой мультиагентной цепочки фактически не получится: основной агент просто будет выполнять все роли самостоятельно.

Создаём рабочую папку

Теперь создадим отдельную папку для практики.

На Windows я открываю Проводник сочетанием `Win + E`.

Выбираю любую удобную учебную директорию, в которой есть права на создание файлов.

Внутри неё создаю папку:

```
hermes-multiagent
```

Теперь открываю эту папку в PowerShell и сохраняю её фактический путь в переменную:

```
$projectRoot = (Get-Location).Path
```

Переменная понадобится в командах и промптах ниже. Её значение зависит от выбранного расположения и не требуется записывать в методичку.

Теперь возвращаюсь в Hermes Desktop.

В центральный чат отправляю:

```
Создай Desktop Project с названием dependency-lab и привяжи его к выбранной папке проекта.
```

```
Переключи текущую сессию в этот Project.
```

```
Никаких файлов пока не создавай и существующие файлы не изменяй.
```

Hermes Desktop имеет специальный механизм Projects. Его инструмент создания проекта принимает путь к каталогу и переводит текущий чат в этот workspace.

После выполнения я открываю справа File Browser и проверяю, что рабочая область соответствует нашей папке.

Даже если рабочая папка визуально определилась правильно, при передаче задачи субагентам указываем значение `$projectRoot`. Это важно, потому что каждый субагент получает отдельную терминальную сессию и не должен угадывать, где находится проект.

Создаём исходный проект

Теперь подготовим состояние проекта до миграции.

Я отправляю основному агенту следующий запрос. На данном этапе мы ещё не создаём субагентов — это просто подготовка лабораторного стенда.

В текущем Project создай исходный учебный Python-проект.

Это только подготовка стенда. Ничего не модернизируй и не исправляй.

Создай следующие файлы с указанным содержимым.

requirements.txt

```
pydantic==1.10.24
```

.gitignore

```
.venv/  
__pycache__/  
*.pyc
```

models.py

```
from pydantic import BaseModel, validator
```

```
class User(BaseModel):  
    name: str  
    email: str  
    age: int  
  
    @validator("age")  
    def validate_age(cls, value):  
        if value < 18:  
            raise ValueError("Возраст должен быть не меньше 18 лет")  
        return value
```

services.py

```
from models import User
```

```
def serialize_user(data: dict) -> dict:  
    user = User.parse_obj(data)  
    return user.dict()
```

main.py

```
from services import serialize_user
```

```
if __name__ == "__main__":
    data = {
        "name": "Анна",
        "email": "anna@example.com",
        "age": 25
    }

    print(serialize_user(data))
```

tests/test_services.py

```
import unittest
```

```
from pydantic import ValidationError
```

```
from services import serialize_user
```

```
class UserServiceTests(unittest.TestCase):
```

```
    def test_valid_user(self):
        data = {
            "name": "Анна",
            "email": "anna@example.com",
            "age": 25
        }

        self.assertEqual(serialize_user(data), data)
```

```
    def test_minor_user_rejected(self):
        data = {
            "name": "Иван",
            "email": "ivan@example.com",
            "age": 16
        }

        with self.assertRaises(ValidationError):
            serialize_user(data)
```

```
if __name__ == "__main__":
    unittest.main()
```

TASK.md

Задача миграции

Проект использует Pydantic 1.x.

Необходимо перейти на Pydantic 2.x.

Требования:

- изменить зависимость на `pydantic>=2,<3`;
- заменить устаревшие API Pydantic v1 на рекомендуемые API Pydantic v2;
- сохранить сигнатуру `serialize_user(data)`;
- сохранить внешний результат работы функции;
- сохранить проверку минимального возраста;
- существующие тесты должны проходить;
- не добавлять новые runtime-зависимости;
- не выполнять несвязанный рефакторинг.

После создания файлов остановись.

После ответа Hermes я справа открываю File Browser и проверяю структуру.

У нас должно получиться:

```
hermes-multiagent/  
  requirements.txt  
  .gitignore  
  TASK.md  
  models.py  
  services.py  
  main.py  
  tests/  
    test_services.py
```

Теперь у нас есть реальный небольшой проект с устаревшим API.

Подготавливаем Python и проверяем исходное состояние

Открываю правую панель Terminal. Если она скрыта, нажимаю:

```
Ctrl + `
```

Сначала проверяю текущую директорию.

В PowerShell:

```
Get-Location
```

Если мы находимся не в папке проекта, перехожу в неё. Например, если в текущей консоли сохранена переменная `$projectRoot`, выполняю:

```
Set-Location $projectRoot
```

Для этого упражнения желательно использовать Python 3.11, 3.12 или 3.13. Pydantic 1.10.24 имеет поддержку Python 3.13, но ветка Pydantic v1 уже не рассчитана на Python 3.14 и выше.

Я создаю виртуальное окружение на Python 3.13:

```
py -3.13 -m venv .venv
```

Если Python 3.13 является обычной командой `python`, можно использовать:

```
python -m venv .venv
```

Теперь я не обязательно активирую окружение. Для учебной работы даже удобнее явно обращаться к его интерпретатору:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

После установки проверяем исходную версию:

```
.\.venv\Scripts\python.exe -c "import pydantic; print(pydantic.VERSION)"
```

Сейчас должна отображаться ветка:

```
1.10...
```

Запускаю тесты:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

На данном этапе тесты должны проходить.

Это важно. Если тесты не проходят ещё до миграции, мы потом не сможем понять, сломал их Coder или проблема существовала изначально.

Создаём Git-baseline

Теперь нам нужно сохранить исходное состояние. Reviewer потом сможет сравнить результат с этой точкой.

В том же терминале выполняю:

```
git init
```

Если на компьютере ещё не настроены данные Git, задаю их только для этого проекта:

```
git config user.name "Student"  
git config user.email "student@example.local"
```

Добавляю файлы:

```
git add .
```

Создаю первый commit:

```
git commit -m "Baseline: Pydantic v1"
```

Теперь проверяю:

```
git status
```

Я хочу увидеть чистое рабочее дерево.

Именно от этой точки мы будем оценивать действия Coder.

Запускаем первый субагент — Planner

Теперь начинается сама мультиагентная часть.

Я возвращаюсь в центральный чат Hermes.

Отправляю:

```
Начинаем мультиагентную цепочку для проекта. Путь проекта: значение переменной $projectRoot.
```

```
Создай через delegate_task отдельного leaf-субагента с ролью Planner.
```

```
Передай Planner весь необходимый контекст самостоятельно.
```

Задача Planner:

изучить проект и TASK.md;
изучить requirements.txt, models.py, services.py, main.py и тесты;
определить, какие изменения необходимы для перехода с Pydantic 1.x на Pydantic 2.x;
составить конкретный план миграции по файлам;
указать, какие проверки необходимо выполнить после изменения.

Planner НЕ ИМЕЕТ ПРАВА изменять файлы, устанавливать новые версии пакетов или выполнять миграцию.

Он должен только изучить проект и вернуть план.

Основной агент также не должен изменять файлы.

После получения результата Planner покажи мне его итоговый план и остановись.

После отправки запроса я смотрю не только на текст ответа.

Над полем ввода должна появиться панель **Subagents**.

Я нажимаю на неё.

Открывается список активных дочерних агентов. В нём я нахожу задачу Planner.

Нажимаю на неё и смотрю активность.

Hermes Desktop позволяет открыть активного субагента и видеть его работу. Там же доступны команды **Steer** и **Stop**. **Steer** позволяет передать работающему субагенту дополнительное указание, а **Stop** прерывает его.

Я здесь специально обращаю внимание студентов на одну вещь. Planner может читать файлы и выполнять диагностические команды, но в Git не должно появиться изменений.

После завершения Planner я возвращаюсь в Terminal и проверяю:

```
git status --short
```

Результат должен быть пустым.

Если появились изменённые файлы, значит разделение ролей нарушено: Planner уже начал выполнять функции Coder.

Анализируем результат Planner

Теперь я смотрю на итоговое сообщение Planner.

У хорошего Planner должен появиться конкретный план, связанный именно с нашим проектом.

Он должен заметить, что в **requirements.txt** используется ветка Pydantic 1.

В `models.py` он должен обнаружить старый механизм `validator`.

В `services.py` должны быть замечены `parse_obj()` и `dict()`.

Он должен связать эти места с миграцией на API второй версии.

Но сейчас мы ничего из этого вручную не исправляем.

Мне здесь важно показать сам принцип передачи работы между агентами.

Planner завершил работу и вернул итог основному агенту. Теперь этот результат должен стать частью контекста следующего субагента.

Поскольку Coder начнёт с чистого контекста, фраза «сделай то, что предложил Planner» сама по себе плохая. Coder не знает историю Planner автоматически. Родитель должен явно передать ему план, путь к проекту, требования и способ проверки. Это базовое правило делегирования в Hermes.

Запускаем второй субагент — Coder

Теперь отправляю основному Hermes второй запрос:

Продолжаем ту же мультиагентную цепочку.

Создай через `delegate_task` НОВОГО отдельного leaf-субагента с ролью Coder.

Не выполняй миграцию основным агентом.

Передай Coder в его context:

1. Путь проекта: значение переменной `$projectRoot`.
2. Полное содержание задачи из `TASK.md`.
3. Итоговый план, который вернул предыдущий Planner.
4. Информацию о том, что `baseline` находится в `Git` и перед началом работы рабочее дерево было чистым.
5. Команду проверки:
`.\.venv\Scripts\python.exe -m unittest discover -s tests -v`

Задача Coder:

выполнить миграцию проекта с `Pydantic 1.x` на `Pydantic 2.x`;

обновить `requirements.txt` на диапазон `pydantic>=2,<3`;

адаптировать используемые API `Pydantic`;

не изменять внешний контракт `serialize_user(data)`;

```
не делать посторонний рефакторинг;  
  
после изменений обновить пакет в .venv;  
  
запустить существующие тесты;  
  
запустить main.py;  
  
проверить git diff;  
  
вернуть итоговый отчёт с перечнем изменённых файлов и результатами проверок.  
  
После завершения Coder ничего больше не запускай.  
Покажи мне его итоговый отчёт и остановись.
```

Снова смотрю на область над полем ввода.

Появляется новый субагент.

Я раскрываю **Subagents** и выбираю Coder.

Теперь поведение должно заметно отличаться от Planner. Coder уже имеет право работать с файлами и терминалом.

Я могу одновременно открыть File Browser справа и наблюдать, какие файлы меняются.

Обычно здесь должны затрагиваться как минимум:

```
requirements.txt  
models.py  
services.py
```

Но я заранее не заставляю Coder менять конкретные строки. Он должен использовать план Planner и самостоятельно выполнить миграцию.

Если я замечаю, что Coder начал переписывать совершенно несвязанные части проекта, могу нажать:

Steer

и передать ему:

```
Не выходи за границы TASK.md. Не выполняй несвязанный рефакторинг.
```

Это хороший момент для демонстрации контроля над работающим субагентом.

Проверяем работу Coder вручную

После завершения Coder я не верю его сообщению только потому, что в нём написано «всё работает».

Открываю Terminal.

Выполняю:

```
git status --short
```

Теперь изменения уже должны быть.

Смотрю разницу:

```
git diff
```

Проверяю установленную версию:

```
.\.env\Scripts\python.exe -c "import pydantic; print(pydantic.VERSION)"
```

Теперь должна использоваться вторая основная версия Pydantic.

Запускаю тесты:

```
.\.env\Scripts\python.exe -m unittest discover -s tests -v
```

Запускаю само приложение:

```
.\.env\Scripts\python.exe main.py
```

Но даже если всё выглядит хорошо, commit пока не создаём.

Нам нужен Reviewer, который увидит незакоммиченный diff и оценит его независимо.

Запускаем третий субагент — Reviewer

Теперь запускаем третье звено.

В центральном чате отправляю:

```
Продолжаем мультиагентную цепочку.
```

```
Planner уже завершил анализ.
```

```
Coder уже внёс изменения.
```

Теперь создай через `delegate_task` НОВОГО отдельного leaf-субагента с ролью `Reviewer`.

`Reviewer` должен получить свежий контекст.

Передай ему:

путь проекта: значение переменной `$projectRoot`;

исходную задачу из `TASK.md`;

информацию о том, что `baseline` сохранён в `Git`;

команду тестирования:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

`Reviewer` должен:

изучить `git diff` относительно `baseline`;

проверить `requirements.txt`;

проверить изменения `models.py` и `services.py`;

убедиться, что миграция действительно выполнена на `Pydantic 2 API`;

проверить, что внешний контракт `serialize_user(data)` сохранён;

проверить, что не выполнен лишний рефакторинг;

самостоятельно запустить тесты;

самостоятельно запустить `main.py`;

проверить установленную `major`-версию `Pydantic`;

сформировать результат `PASS` или `FAIL`;

перечислить найденные замечания.

`Reviewer` НЕ ДОЛЖЕН изменять никакие файлы и НЕ ДОЛЖЕН исправлять проблемы.

Основной агент также ничего не исправляет.

После отчёта `Reviewer` остановись.

Теперь снова нажимаю на панель `Subagents`.

Открываю `Reviewer`.

Здесь мне особенно важно, чтобы это действительно был новый дочерний агент.

Reviewer не должен продолжать рассуждение Coder. Он получает свежий контекст и смотрит на результат со стороны.

Это одна из причин, почему отдельный субагент подходит для review: Hermes изолирует его историю от предыдущего исполнителя, а значит Reviewer не обязан защищать решения, которые сам же ранее принимал. Механизм `/review` в Hermes построен на похожей идее отдельного reviewer-субагента, но в этой лабораторной мы сознательно используем обычный `delegate_task`, чтобы явно построить именно цепочку Planner → Coder → Reviewer.

Что должно произойти на этапе Reviewer

Я ожидаю от Reviewer не просто сообщение «код хороший».

Он должен проверить реальные файлы.

Он должен посмотреть Git diff.

Он должен выполнить тесты.

Он должен проверить версию установленной библиотеки.

И только после этого сформировать итог.

Если Reviewer сообщает:

PASS

мы смотрим, на каких доказательствах это основано.

Если он сообщает:

FAIL

мы также не исправляем проблему автоматически. Для текущей практики это важный результат: независимый reviewer нашёл недостаток после работы coder.

Если бы мы строили более сложную агентную систему, после `FAIL` можно было бы организовать дополнительный цикл Reviewer → Coder → Reviewer. Но это уже другая архитектура. Сейчас наша цепочка заканчивается на первом независимом review.

Проверяем, что Reviewer ничего не изменил

После завершения Reviewer снова открываю Terminal.

Выполняю:

```
git status --short
```

Затем:

```
git diff
```

Изменения должны соответствовать работе Coder.

Reviewer не должен создавать дополнительные изменения.

Если после Reviewer появляются новые правки исходного кода, значит мы неправильно сформулировали его роль или он вышел за установленные ограничения.

Это хороший пример того, зачем в мультиагентных системах недостаточно просто дать агенту имя `Reviewer`. Роль должна определяться не названием, а конкретными разрешёнными и запрещёнными действиями.

Проверяем дерево субагентов в Hermes

Теперь я хочу посмотреть цепочку целиком.

В чате Hermes ввожу:

```
/agents
```

Команда `/agents`, она же `/tasks`, открывает представление активных и завершённых субагентов. В Hermes здесь можно просматривать структуру делегирования, состояние дочерних агентов и историю их работы.

Я хочу увидеть три разных выполнения.

Planner.

Coder.

Reviewer.

Если вместо этого вся работа была сделана основным агентом, наша учебная задача фактически не выполнена.

Если Coder и Reviewer запускались одновременно, цепочка также построена неправильно. Reviewer должен получать уже готовый результат Coder.

Если Planner и Coder запускались одновременно, Coder не мог использовать итоговый план Planner.

Поэтому в этой задаче последовательность принципиальна.

Теперь автоматизируем всю цепочку

До этого момента я специально запускал три этапа отдельными командами. Это нужно было для того, чтобы мы увидели механику.

Теперь можно показать, как ту же цепочку передать основному агенту как единую цель.

Перед повторным экспериментом я сначала возвращаю проект к baseline.

В Terminal выполняю:

```
git restore .
```

Проверяю:

```
git status --short
```

Рабочее дерево снова должно быть чистым.

Возвращаю старую зависимость в виртуальное окружение:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Теперь в чат отправляю единый оркестрационный запрос.

```
/goal Выполни учебную миграцию проекта, путь которого передан как значение переменной $projectRoot, строго через последовательную мультиагентную цепочку Planner → Coder → Reviewer.
```

Основной агент является только координатором и не должен самостоятельно изменять исходные файлы.

Для каждого этапа обязательно создавай новый отдельный leaf-субагент через `delegate_task`.

Этап 1 – Planner.

Передай Planner путь проекта и `TASK.md`.

Planner должен изучить проект, составить план миграции и определить проверки.

Planner не имеет права изменять файлы.

Дождись завершения Planner и получи его итоговый результат.

Только после этого переходи к следующему этапу.

Этап 2 – Coder.

Создай нового субагента.

Передай ему путь проекта, TASK.md и итоговый план Planner.

Coder должен обновить зависимость с Pydantic 1.x до Pydantic 2.x, адаптировать код, установить обновлённые зависимости и выполнить проверки.

Команда тестов:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Coder должен вернуть перечень изменений и результаты проверок.

Дождись полного завершения Coder.

Только после этого запускай следующий этап.

Этап 3 – Reviewer.

Создай нового независимого субагента.

Передай ему путь проекта, исходные требования и информацию о baseline в Git.

Не передавай ему рассуждения Coder как истину.

Reviewer должен самостоятельно изучить git diff, проверить миграцию, версию Pydantic, тесты и запуск main.py.

Reviewer не имеет права изменять файлы.

Reviewer должен вернуть PASS или FAIL и конкретные замечания.

Не запускай Planner, Coder и Reviewer параллельно.

Не заменяй какой-либо этап собственной работой основного агента.

После завершения всех трёх субагентов сформируй только итоговую сводку:

```
что решил Planner;  
что изменил Coder;  
что проверил Reviewer;  
финальный статус Reviewer.
```

Даже если Reviewer вернёт FAIL, автоматически проблему не исправляй. На этом цепочка должна завершиться.

Здесь мы используем `/goal` не вместо multi-agent механизма, а для оркестрации процесса между несколькими ходами. В Hermes постоянная цель сохраняется между ходами, а после каждого ответа система проверяет, достигнута ли она; если нет, Hermes может автоматически продолжить работу без нашего сообщения «продолжай».

То есть `/goal` поддерживает жизнь всей цепочки, а `delegate_task` создаёт собственно исполнителей.

Это две разные функции.

Наблюдаем автоматическую цепочку

После отправки большого запроса я не переключаюсь сразу на что-то другое.

Смотрю на `Subagents`.

Сначала должен появиться Planner.

Пока Planner не завершился, Coder появляться не должен.

После Planner должен появиться Coder.

После завершения Coder должен появиться Reviewer.

Если я вижу одновременно три активных субагента, значит основной агент не выполнил наше требование о последовательности.

Для этой задачи это ошибка архитектуры, даже если итоговый код окажется правильным.

Мы специально изучаем зависимости между этапами.

Planner создаёт результат, необходимый Coder.

Coder создаёт результат, необходимый Reviewer.

Поэтому здесь синхронная последовательность важнее потенциального выигрыша от параллельного запуска.

Что смотрим в отчёте Planner

Теперь мы разбираем полученные результаты уже не как ответы нейросети, а как результаты отдельных ролей.

У Planner я проверяю, нашёл ли он все места, связанные с Pydantic v1.

Мне важно, чтобы его план был привязан к конкретным файлам.

Формулировка вроде:

Нужно обновить код под новую версию.

практически бесполезна.

Хороший план должен позволять следующему агенту понять область работы без повторного полного исследования задачи.

Что смотрим в отчёте Coder

У Coder я проверяю не красоту объяснения, а реальные изменения.

Открываю:

```
git diff
```

Проверяю, соответствует ли diff задаче.

Если изменилось двадцать несвязанных файлов, задаю вопрос: почему?

Если Coder изменил только необходимые участки, это хороший признак.

После этого снова выполняю:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

И:

```
.\.venv\Scripts\python.exe main.py
```

Что смотрим в отчёте Reviewer

Здесь самый важный вопрос звучит так: проверил ли Reviewer результат или просто пересказал изменения?

Фразы:

```
Изменения выглядят корректно.
```

недостаточно.

Я хочу видеть, что Reviewer действительно выполнил команды проверки.

Он должен самостоятельно подтвердить версию библиотеки.

Он должен посмотреть diff.

Он должен прогнать тесты.

Только после этого его **PASS** приобретает смысл.

Финальная фиксация результата

Если Reviewer вернул **PASS**, я могу зафиксировать миграцию.

В Terminal выполняю:

```
git add .
```

Затем:

```
git commit -m "Migrate project to Pydantic v2"
```

Проверяю:

```
git status
```

В конце рабочее дерево должно быть чистым.

Теперь у нас существует понятная история:

```
Baseline: Pydantic v1
```

и затем:

```
Migrate project to Pydantic v2
```

Это позволяет показать студентам, что агентная работа происходит внутри обычного процесса разработки, а не существует отдельно от Git, тестов и проекта.

Что именно мы собрали

В результате этой практики у нас появился основной Hermes Agent, который выполняет роль координатора.

Planner получает свежий контекст, анализирует исходное состояние и возвращает план.

Coder получает задачу вместе с планом и выполняет фактическую миграцию.

Reviewer начинает отдельную проверку уже изменённого проекта и возвращает независимое заключение.

Главный агент связывает эти результаты между собой, но не обязан самостоятельно выполнять работу каждого исполнителя.

Именно здесь мы практически видим отличие single-agent режима от multi-agent. Если бы мы передали обычному агенту запрос «обнови Pydantic и проверь результат», возможно, он решил бы задачу быстрее. Но мы не увидели бы разделения ответственности и независимого контекста.

В нашей схеме специально появляется граница между планированием, реализацией и проверкой. Это и есть основная учебная ценность цепочки Planner → Coder → Reviewer.